DISEÑO SOFTWARE

MODELADO

Un modelo es una simplificación de la realidad que permite comprender mejor el sistema que estamos desarrollando.

¿Por qué modelamos?

Los sistemas son lo suficientemente complejos como para no poder comprenderlos en su totalidad.

El modelado es una técnica de divide y vencerás que nos permite centrarlos en un único aspecto del problema en estudio a la vez.

Principios básicos del modelado

La elección de qué modelos crear tiene una profunda influencia sobre cómo se acomete un problema y cómo se da forma a una solución.

- Todo modelo puede ser expresado a diferentes niveles de precisión.
- Los mejores modelos están ligados a la realidad.
- Un único modelo no es suficiente. Cualquier sistema no trivial se aborda mejor a través de un pequeño conjunto de modelos casi independientes.

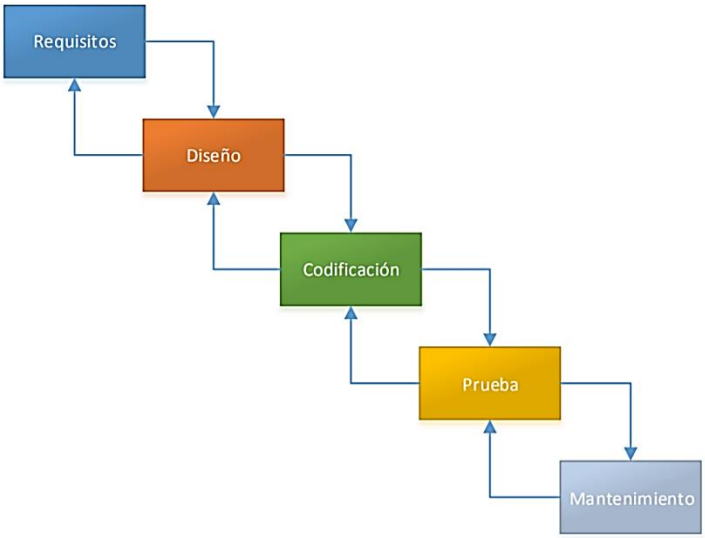
INGENIERÍA DEL SOFTWARE

Es la aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento del software, es decir, la aplicación de la ingeniería al software.

En la ingeniería del software se describen distintos modelos de desarrollo del software. Estos modelos consisten básicamente en el establecimiento de unas fases separadas del proceso con objetivos y tareas propias: requisitos, análisis, diseño, etc.

Modelo en cascada

El desarrollo se divide en una serie de fases secuenciales. Existe una fase formal de análisis de requisitos previa al diseno y a la codificación.



@informaticapau

PROCESO UNIFICADO

- Sigue un proceso iterativo e incremental.
- El desarrollo se divide en fases: Inicio, Elaboración, Construcción y Transición.
- En cada una de estas fases se repiten una o varias iteraciones.
- En cada iteración se llevan a cabo (con distinto grado de esfuerzo) las disciplinas típicas de la ingeniería del software: requisitos, análisis, diseño, etc.

Disciplinas del Proceso Unificado

- **Modelado del negocio**: Se describe la estructura y dinámica de la organización objetivo identificando sus problemas actuales y sus posibles soluciones.
- **Requisitos**: El objetivo es describir qué es lo que tiene que hacer el sistema y poner a los desarrolladores y al cliente de acuerdo en esta descripción.
- **Análisis y Diseño**: Describe la estructura y comportamiento del software en base a los requisitos capturados adaptando el diseño al entorno de implementación.
- **Implementación**: Se implementan las clases y objetos en términos de componentes (ficheros fuente, binarios, ejecutables, etc.).
- **Prueba**: Se comprueba que el funcionamiento es correcto analizando diversos aspectos: los objetos como unidades, la integración entre objetos, etc.
- **Despliegue**: Se crea la versión externa del producto, se empaqueta, se distribuye y se instala en el lugar de trabajo y se da asistencia a los usuarios.

@informaticapau

PROGRAMACIÓN ORIENTADA A OBJETOS

Formada por objetos con un comportamiento y estado propios que se comunican intercambiando mensajes.

OBJETOS Y TIPOS ABSTRACTOS DE DATOS (TADS)

Similitudes entre Objetos y TADs

En principio un objeto no es más que un tipo abstracto de datos. Simplemente se cambia la notación y en vez de hablar de tipos y variables hablamos de clases y objetos.

Un objeto pertenece a una clase, mantiene un estado interno privado y ofrece al exterior procedimientos y funciones (comúnmente denominados métodos) que permiten cambiar dicho estado.

Los objetos mantienen características ya existentes en los TADs como abstracción, encapsulación, modularidad, etc.

¿Qué añaden los objetos?

Una nueva filosofía de diseño: Pasamos de un diseño *Top-Down* a un diseño *Bottom-Up*.

Herencia: Los objetos pueden heredar atributos y comportamiento definidos en otros objetos. A la herencia se añaden otras características asociadas como el polimorfismo o la ligadura dinámica.

DISEÑO TOP-DOWN

Consiste en ir descomponiendo el programa sistemáticamente en partes más pequeñas y manejables. Es el mecanismo típico de diseño de programas estructurados. El proceso de división continua hasta que las partes son tan simples que pueden ser desarrolladas directamente, sin necesitar más descomposición.

Problemas del diseño Top-Down

El énfasis se pone en desarrollar las funciones que permitan resolver el problema, pero no en el diseño de las estructuras de datos necesarias.

Tiende a producir diseños que son únicos para el problema que se está planteando, ya que las partes de más bajo nivel son desarrolladas únicamente para cubrir las necesidades de las partes de más alto nivel.

Es muy complicado reusar trabajo realizado en un proyecto para otro proyecto.

DISEÑO BOTTOM-UP

Consiste en comenzar el diseño identificando las estructuras básicas de datos del programa, su comportamiento y sus interacciones. El programa se construye combinando entre sí estos bloques básicos. Es el mecanismo típico de diseño en la orientación a objetos. Los bloques básicos del código son objetos “autosuficientes” que encapsulan su propio estado interno y responden a mensajes. Los objetos identificados pueden fácilmente ser reusables en problemas similares.

@informaticapau

HERENCIA

Relación “ES UN” entre clases, en las que una clase comparte la estructura de comportamiento definida en una o más clases.

La herencia permite definir nuevas clases que hereden las características de sus clases padre y añadan, a su vez, características especializadas propias. También puede usarse para generalizar en una superclase características comunes existentes en las subclases.

ASIGNACIÓN DE RESPONSABILIDADES

Consiste en identificar responsabilidades (que podemos identificar como contratos u obligaciones) y asignárselas a los objetos.

Es una metáfora que asemeja a los objetos software como entidades con responsabilidades que colaboran con otras entidades para realizar el trabajo encomendado.

Una responsabilidad no es un método, los métodos se utilizan para cumplir las responsabilidades.

Incrementa la encapsulación ya que un cliente no tiene por qué saber cómo un servicio cumplimenta su responsabilidad.

¿Cómo le asignamos responsabilidades a los objetos?

Criterio general: Debemos asignar una responsabilidad a la clase que tiene la información necesaria para llevarla a cabo.

Una responsabilidad necesita información para poder ser llevada a cabo (acerca de otros objetos, acerca del propio objeto que la lleva a cabo, acerca de lo que el propio objeto puede obtener, etc.).

VENTAJAS DE LA ORIENTACIÓN A OBJETOS

- **Reutilización**: Los objetos bien diseñados pueden utilizarse como base de otros sistemas, que se constituyen como una combinación de los objetos existentes.
- **Modularidad**: Los objetos son autocontenidos y tienen definido de forma clara sus interfaces con otros objetos.
- **Comprensión**: Al estar los datos y procedimientos que conforman los objetos encapsulados en un mismo compartimento, los objetos pueden ser desarrollados y probados en forma independiente.
- **Naturalidad**: El análisis y el diseño que dividen un dominio en objetos, está más acorde con la realidad que una descomposición funcional por refinamiento progresivo.
- **Extensibilidad**: La herencia permite que sea posible definir y utilizar de forma clara módulos funcionalmente incompletos y, luego, permiten su extensión sin trastornar la operación de los módulos cliente.
- **Escalabilidad**: Los diseños orientados a objetos, al ser fácilmente extensibles, permiten que el esfuerzo no aumente exponencialmente con el tamaño y la complejidad del proyecto, tal y como sucede con los sistemas convencionales.

@informaticapau

INCONVENIENTES DE LA ORIENTACIÓN A OBJETOS

- **Curva de aprendizaje**: Aprender un lenguaje orientado a objetos y dominar sus técnicas fundamentales es más complejo que aprender un lenguaje imperativo.
- **Cambio de enfoque**: Los desarrolladores generalmente aprenden a programar con lenguajes imperativos a través de una filosofía Top-Down y deben cambiar a una filosofía Bottom-Up.
- **Reutilizaciones ineficientes**: Los objetos en sí mismos no son reutilizables por ser objetos, hay que programar pensando en la reutilización y no siempre se invierte el esfuerzo necesario.
- **Orientación a objetos forzada**: "Si la única herramienta que tienes es un martillo, entonces tiendes a ver todos los problemas como si fueran clavos". Forzar la utilización de objetos en determinados entornos puede producir diseño no adecuados (ej. la clase Math no sigue la OO).

@informaticapau